Sistema de gestión – Reserva Nacional Pampa Galeras

Oct 25, 2026 · @Grupocinco

Resumen y arquitectura

El sistema gestiona la Reserva Nacional Pampa Galeras Bárbara D'Achille: su fauna y flora, los animales registrados por hato y las reservas de visita de los turistas a las actividades diarias. Tiene tres capas, y toda la lógica de base de datos está escrita en SQL, sin PL/SQL.

Capa	Tecnología	Carpeta	Qué hace
Interfaz web	React 18 + Vite	`frontend/`	Pantallas de consulta y gestión; habla con la API por HTTP
API	Node.js + Express, `oracledb`	`backend/`	Valida datos, controla el acceso con JWT y ejecuta las transacciones SQL
Base de datos	Oracle Database 21c XE (PDB `XEPDB1`)	`proyecto_completo.sql`	Tablas, restricciones, vistas, usuarios, roles y datos iniciales

El backend se conecta como `ADMIN_RNPG`, dueño de todas las tablas. Las fotos que se suben se guardan en `backend/uploads/` y la base solo almacena su ruta.

Estructura principal del repositorio:

- `proyecto_completo.sql`: crea toda la base de datos y se ejecuta como SYS.
- `backend/src/index.js`: arranque del servidor y registro de rutas.
- `backend/src/routes/`: un archivo por recurso (animales, especies, hatos, reservas, actividades, reservasVisita, turistas, auth, execute).
- `backend/src/db.js`: conexión a Oracle y hora de Perú.
- `frontend/src/pages/`: una pantalla por módulo.
- `frontend/src/utils.js`: funciones compartidas (formato de fechas, horarios, nombres de clases).

Funcionalidades por módulo

Cualquier visitante puede consultar animales, especies, actividades y áreas; registrar, editar o eliminar algo exige iniciar sesión como administrador. Turistas y reservas solo se ven con sesión iniciada.

Módulo	Ruta	Qué permite
Inicio	`/`	Presentación de la reserva y accesos a cada sección
Animales	`/animales`	Listar, buscar, registrar, editar y eliminar animales; subir una foto. Cada tarjeta muestra el nombre común y científico de la especie, la clase, el sexo, la edad, el hato y las notas
Especies	`/especies`	Pestañas Fauna y Flora; fauna agrupada por clase y flora por familia; buscador; registrar, editar y eliminar especies con su reino y estado de conservación; ver cuántos animales tiene cada una
Actividades	`/actividades`	Registrar, editar y eliminar actividades (nombre, tipo, descripción, lugar, duración, costo, foto) y el cupo por horario; cada actividad se ofrece todos los días a las 08:00, 11:00 y 14:00
Reservas	`/reservas`	Crear reservas eligiendo actividad, fecha, horario, turista y número de personas; ver cupos libres por horario; confirmar o cancelar; filtrar por estado y por fecha
Turistas	`/turistas`	Registrar, buscar, editar y eliminar turistas (DNI, nombre, apellido, edad, nacionalidad); ver cuántas reservas activas tiene cada uno
Áreas	`/areas`	Registrar, editar y eliminar las áreas protegidas (nombre, ubicación, superficie, categoría, foto)
Admin	`/admin`	Inicio de sesión y una consola SQL de desarrollo, activa solo con `ALLOW_EXECUTE=true`

Reglas visibles para el usuario en Reservas:

- Se puede reservar desde hoy hasta 60 días después.
- Cada horario muestra sus cupos libres, "Agotado" o "Ya pasó".
- El total a pagar es el costo por persona multiplicado por el número de personas.

- Una reserva nueva queda PENDIENTE; luego se confirma o se cancela, y al cancelar sus cupos vuelven a quedar libres.
- Una visita cuya fecha y hora ya pasaron no se puede modificar.

Modelo de datos

La base tiene 13 tablas en el esquema `ADMIN_RNPG`. Todas las claves primarias son columnas `IDENTITY`, así que Oracle genera el ID al insertar.

Tabla	Para qué sirve	Columnas principales	Restricciones destacadas	Filas iniciales
RESERVA	Áreas protegidas	`id_reserva`, `nombre`, `ubicacion`, `superficie_ha`, `categoria`, `foto_ruta`	`superficie_ha >= 0`	2
HATO	Grupos de animales por zona de un área	`id_hato`, `id_reserva`, `cantidad_animales`, `zona`	FK a RESERVA con borrado en cascada	2
ESPECIE	Catálogo de fauna y flora	`id_especie`, `nombre_comun`, `nombre_cientifico`, `clase`, `orden`, `familia`, `estado_conservacion`, `reino`	`reino` FLORA o FAUNA; estado LC, NT, VU, EN, CR, DD o NE; único `(id_especie, reino)`	294 (125 fauna, 169 flora)
ANIMAL	Individuos registrados	`id_animal`, `id_hato`, `id_especie`, `sexo`, `edad_anios`, `fecha_registro`, `observaciones`, `foto_ruta`, `reino_especie`	`sexo` M, F o U; `edad_anios >= 0`; `reino_especie` siempre FAUNA; FK compuesta a ESPECIE	14
AUDITORIA_ANIMAL	Registro de operaciones sobre animales	`id_auditoria`, `id_animal`, `operacion`, `usuario`, `fecha`	Ninguna FK; la aplicación aún no escribe en ella	0
TURISTA	Visitantes	`id_turista`, `id_reserva`, `dni`, `nombre`, `apellido`, `edad`, `nacionalidad`	`dni` único de 8 caracteres; edad entre 0 y 130	3

Tabla	Para qué sirve	Columnas principales	Restricciones destacadas	Filas iniciales
ACTIVIDAD	Actividades turísticas	`id_actividad`, `nombre`, `tipo`, `lugar`, `duracion_min`, `costo_soles`, `descripcion`, `foto_ruta`	Duración y costo no negativos; sin fecha, porque son diarias	5
HORARIO_ACTIVIDAD	Los 3 turnos de cada actividad	`id_horario`, `id_actividad`, `hora_inicio`, `cupo_maximo`	Hora 08:00, 11:00 o 14:00; único `(id_actividad, hora_inicio)`; cupo mayor a 0	15
RESERVA_VISITA	Reserva de un turista para un horario y una fecha	`id_reserva_visita`, `id_reserva`, `id_turista`, `id_horario`, `fecha_visita`, `num_personas`, `monto_total`, `estado`, fechas de creación, confirmación y cancelación, `motivo_cancelacion`	Estado PENDIENTE, CONFIRMADA o CANCELADA; `fecha_visita` sin hora; personas mayor a 0	0
AUDITORIA_RESERVA	Historial de cambios de estado de cada reserva	`id_auditoria`, `id_reserva_visita`, `operacion`, `estado_anterior`, `estado_nuevo`, `usuario`, `fecha`	FK a RESERVA_VISITA con borrado en cascada; `usuario` toma la sesión de Oracle	0
TURISTA_ACTIVIDAD	Participación de un turista en una actividad en una fecha y hora	`id_turista`, `id_actividad`, `fecha_participacion`, `estado`	PK compuesta de las tres primeras columnas	3
PERSONAL	Guardaparques, guías e investigadores	`id_personal`, `nombre`, `apellido`, `cargo`, `especialidad`	—	5
ACTIVIDAD_PERSONAL	Personal asignado a cada actividad	`id_personal`, `id_actividad`, `rol`, `fecha_asignacion`	PK compuesta `(id_personal, id_actividad)`	5

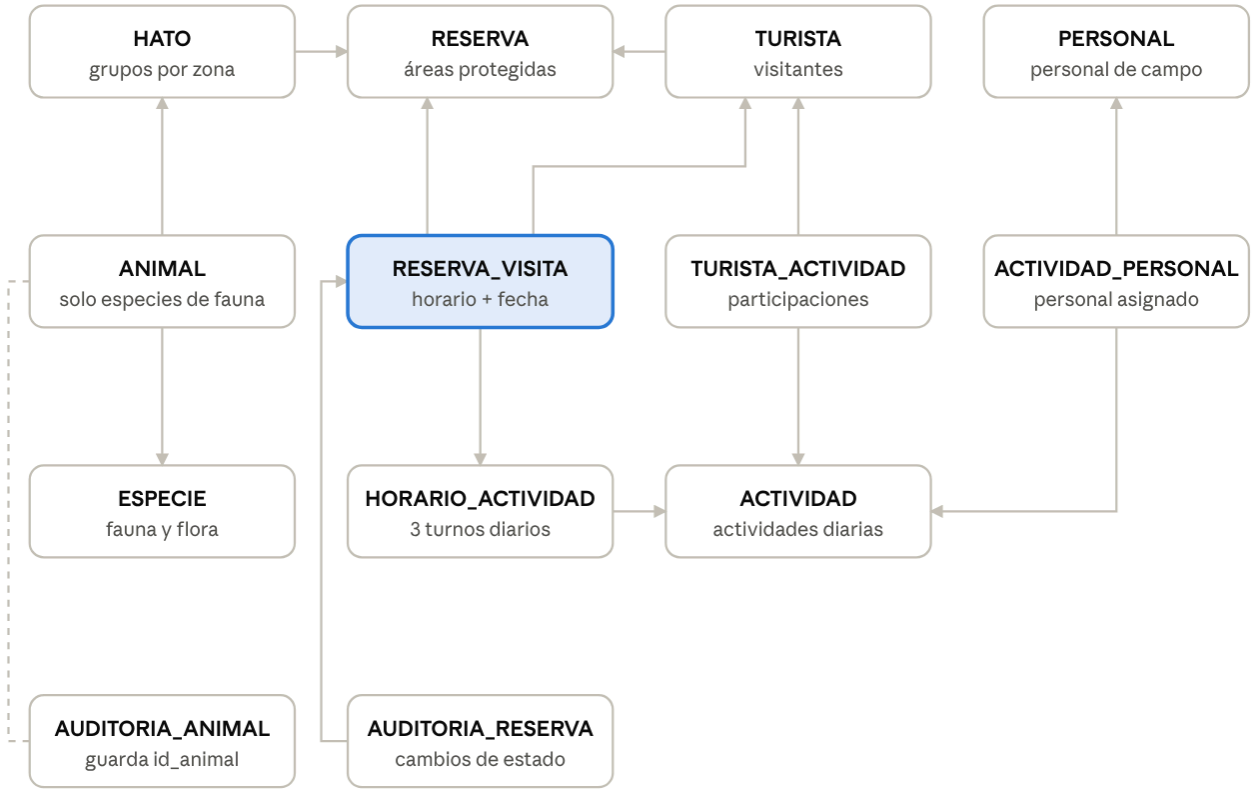
PERSONAL y ACTIVIDAD_PERSONAL tienen datos y permisos, pero todavía no tienen pantalla en la aplicación.

Relaciones

RESERVA_VISITA es el centro del negocio: cada fila une un área (RESERVA), un turista y un horario de una actividad en una fecha concreta.

Cada reserva de visita une un turista, un horario y un área

La flecha va de la tabla hija (muchos) a la tabla padre (uno). Línea punteada: relación sin FK.



Modelo entidad-relación · 13 tablas, 13 claves foráneas

Tabla hija	Tabla padre	Columna	Al borrar el padre
HATO	RESERVA	`id_reserva`	Se borran sus hatos
ANIMAL	HATO	`id_hato`	Se borran sus animales
ANIMAL	ESPECIE	`(id_especie, reino_especie)`	No se permite si hay animales
TURISTA	RESERVA	`id_reserva`	El turista queda sin área (`NULL`)

Tabla hija	Tabla padre	Columna	Al borrar el padre
TURISTA_ACTIVIDAD	TURISTA	`id_turista`	Se borran sus participaciones
TURISTA_ACTIVIDAD	ACTIVIDAD	`id_actividad`	Se borran sus participaciones
ACTIVIDAD_PERSONAL	PERSONAL	`id_personal`	Se borran sus asignaciones
ACTIVIDAD_PERSONAL	ACTIVIDAD	`id_actividad`	Se borran sus asignaciones
HORARIO_ACTIVIDAD	ACTIVIDAD	`id_actividad`	Se borran sus horarios
RESERVA_VISITA	RESERVA	`id_reserva`	No se permite si hay reservas
RESERVA_VISITA	TURISTA	`id_turista`	No se permite si hay reservas
RESERVA_VISITA	HORARIO_ACTIVIDAD	`id_horario`	No se permite si hay reservas
AUDITORIA_RESERVA	RESERVA_VISITA	`id_reserva_visita`	Se borra su historial

Por eso no se puede eliminar una actividad, un turista o un área que ya tenga reservas de visita: Oracle responde ORA-02292 y la aplicación muestra un mensaje claro. TURISTA_ACTIVIDAD y ACTIVIDAD_PERSONAL resuelven relaciones de muchos a muchos. AUDITORIA_ANIMAL guarda el `id_animal` sin clave foránea.

Reglas de integridad y vistas

Las reglas importantes las exige Oracle con restricciones declarativas, así que se cumplen aunque los datos se inserten desde SQL Developer y no desde la aplicación. No hay triggers, procedimientos ni paquetes.

Regla	Cómo se implementa
Un animal solo puede ser de una especie de fauna	`ESPECIE` tiene una clave única `(id_especie, reino)`. `ANIMAL.reino_especie` vale siempre `'FAUNA'` (DEFAULT + CHECK) y su FK apunta a `(id_especie, reino)`. Si la especie es flora, la FK falla con ORA-02291
Una especie con animales no puede pasar a flora ni eliminarse	La misma FK: cambiar `reino` o borrar la fila padre da ORA-02292
Solo existen los turnos 08:00, 11:00 y 14:00, uno de cada uno por actividad	`CHECK (hora_inicio IN ('08:00','11:00','14:00'))` y `UNIQUE (id_actividad, hora_inicio)`
Un turista no puede tener dos reservas activas en el mismo horario y fecha	Índice único basado en función: `CASE WHEN estado IN ('PENDIENTE','CONFIRMADA') THEN … END` sobre turista, horario y fecha. Las canceladas quedan fuera del índice y no bloquean
La fecha de visita no lleva hora	`CHECK (fecha_visita = TRUNC(fecha_visita))`
Estados válidos de una reserva	`CHECK (estado IN ('PENDIENTE','CONFIRMADA','CANCELADA'))`
Valores no negativos o con rango	CHECK en superficie, cupo, personas, monto, duración, costo, edad del animal y edad del turista (0 a 130)
DNI sin duplicados	`UNIQUE (dni)` en TURISTA
Estado de conservación válido	`CHECK` con LC, NT, VU, EN, CR, DD y NE

El cupo disponible no se guarda en una columna: se calcula como `cupo_maximo` menos las personas con reserva activa en ese horario y día. Así nunca queda desactualizado.

Vistas disponibles:

Vista	Qué devuelve	Tablas que une
`vw_ocupacion_horario`	Personas con reserva activa por horario y fecha	RESERVA_VISITA

Vista	Qué devuelve	Tablas que une
`vw_reservas_visita`	Detalle de cada reserva: área, turista, DNI, actividad, lugar, fecha, hora, personas, costo, monto, estado y fechas de cada cambio	RESERVA_VISITA, RESERVA, TURISTA, HORARIO_ACTIVIDAD, ACTIVIDAD
`vw_resumen_actividades`	Por actividad: reservas confirmadas, total de visitantes e ingresos	ACTIVIDAD, HORARIO_ACTIVIDAD, RESERVA_VISITA
`vw_animales_por_reserva`	Animales con su área, hato, especie, sexo, edad y fecha de registro	RESERVA, HATO, ANIMAL, ESPECIE
`vw_especies_conservacion`	Número de especies e individuos por estado de conservación, de más a menos amenazado	ESPECIE, ANIMAL

Usuarios, roles y perfiles de Oracle

El script crea 4 usuarios, cada uno con un rol y un perfil distintos. Todas las tablas pertenecen a `ADMIN_RNPG`, que es también el usuario con el que se conecta el backend.

Usuario	Perfil	Cuota en `users`	Rol	Para qué sirve
`ADMIN_RNPG`	PERFIL_ADMIN	Ilimitada	rol_administrador	Dueño del esquema; lo usa el backend
`APP_RNPG`	PERFIL_APLICACION	50 MB	rol_aplicacion	Operación de una aplicación: leer y modificar datos

Usuario	Perfil	Cuota en `users`	Rol	Para qué sirve
`REPORTES_RNPG`	PERFIL_APLICACION	5 MB	rol_consulta	Solo lectura para reportes
`PERSONAL_SERNANP`	PERFIL_APLICACION	5 MB	rol_personal	Personal de campo: gestiona animales, hatos y personal

Perfiles:

- **PERFIL_ADMIN**: sin límites de sesiones, inactividad, tiempo de conexión, CPU, vida de la contraseña ni intentos fallidos.
- **PERFIL_APLICACION**: máximo 5 sesiones por usuario, 60 minutos de inactividad, 600 minutos de conexión, bloqueo tras 3 intentos fallidos durante 1 día, contraseña válida 30 días y no reutilizable antes de 30 días.

Permisos por rol:

Rol	Privilegios de sistema	Permisos sobre objetos
rol_administrador	CREATE TABLE, VIEW, PROCEDURE, SEQUENCE, TRIGGER y ROLE; ALTER USER y DROP USER, con ADMIN OPTION	Es dueño de todo
rol_aplicacion	SELECT, INSERT y UPDATE ANY TABLE	SELECT, INSERT, UPDATE y DELETE en las tablas base; SELECT, INSERT y UPDATE en RESERVA_VISITA y HORARIO_ACTIVIDAD; SELECT e INSERT en AUDITORIA_RESERVA; SELECT en las 5 vistas
rol_consulta	SELECT ANY TABLE	SELECT en las tablas principales, en RESERVA_VISITA, HORARIO_ACTIVIDAD y en las 5 vistas

Rol	Privilegios de sistema	Permisos sobre objetos
rol_personal	—	SELECT en RESERVA, ACTIVIDAD, TURISTA y ESPECIE; SELECT, INSERT y UPDATE en PERSONAL; SELECT, INSERT, UPDATE y DELETE en ANIMAL y HATO; INSERT en AUDITORIA_ANIMAL; SELECT en las 5 vistas

Los privilegios `ANY TABLE` de rol_aplicacion y rol_consulta permiten acceder a tablas de cualquier esquema. Son cómodos para un curso, pero en producción convendría quitarlos y dejar solo los permisos por objeto.

Transacciones de reserva

Crear, confirmar y cancelar una reserva son transacciones que el backend ejecuta con sentencias SQL en una sola conexión: si cualquier paso falla, hace `ROLLBACK` y no queda nada a medias. Si todo sale bien, hace `COMMIT`. No se usa PL/SQL.

Crear una reserva (queda PENDIENTE):

1. Comprueba que existan el área (RESERVA) y el turista.
2. Bloquea la fila del horario con `SELECT … FROM HORARIO_ACTIVIDAD … FOR UPDATE`. Dos personas que reservan el mismo horario a la vez quedan en fila: la segunda espera a que la primera termine, y así nunca se vende más del cupo.
3. En esa misma consulta obtiene el cupo, el costo de la actividad y si el horario ya pasó o está a más de 60 días.
4. Verifica que el turista no tenga otra reserva activa en ese horario y fecha.
5. Suma las personas ya reservadas en ese horario y día, y rechaza la reserva si no alcanzan los cupos.
6. Inserta en RESERVA_VISITA con `monto_total = costo × personas`, y obtiene el ID nuevo con `RETURNING id_reserva_visita INTO`.
7. Registra la participación en TURISTA_ACTIVIDAD con `MERGE`: la inserta, o la reactiva si ya existía de una reserva cancelada.
8. Inserta en AUDITORIA_RESERVA la operación CREAR.
9. `COMMIT`.

Confirmar (solo desde PENDIENTE) y **cancelar** (desde PENDIENTE o CONFIRMADA):

1. Bloquea la reserva con `SELECT … FOR UPDATE` y lee su estado, turista, horario, fecha y si la visita ya pasó.

2. Rechaza el cambio si el estado no lo permite o si la fecha y hora de la visita ya pasaron.
3. Actualiza el estado y la fecha de confirmación o de cancelación; al cancelar guarda el motivo, que es opcional.
4. Actualiza el estado de la participación en TURISTA_ACTIVIDAD.
5. Inserta en AUDITORIA_RESERVA la operación, con el estado anterior y el nuevo.
6. `COMMIT`.

Al cancelar, los cupos vuelven solos, porque el cálculo de ocupación solo cuenta las reservas PENDIENTE y CONFIRMADA.

Otras transacciones del sistema:

- **Crear actividad**: inserta la actividad y sus 3 horarios con el mismo cupo (10 si no se indica).
- **Editar actividad**: si cambia el cupo, bloquea sus horarios y no permite bajarlo por debajo de las personas ya reservadas en alguna fecha futura.

Todas las comparaciones de fecha y hora usan la hora de Perú: `CAST(SYSTIMESTAMP AT TIME ZONE 'America/Lima' AS DATE)`. Así funcionan igual aunque el servidor Oracle esté en UTC.

API del backend

La API corre en el puerto 4000 y todas sus rutas empiezan con `/api`. Las rutas marcadas "Sí" exigen la cabecera `Authorization: Bearer <token>`, que se obtiene al iniciar sesión y vence a las 2 horas.

Método	Ruta	Token	Qué hace
POST	`/api/auth/login`	No	Recibe `user` y `pass`; si coinciden con `ADMIN_USER` y `ADMIN_PASS`, devuelve el token
GET	`/api/animales`	No	Lista animales con su especie, clase y hato
POST · PUT · DELETE	`/api/animales[/:id]`	Sí	Crear, editar y eliminar un animal
POST	`/api/animales/:id/foto`	Sí	Subir la foto de un animal (la pantalla limita a 5 MB)

Método	Ruta	Token	Qué hace
GET	`/api/especies?reino=FAUNA|FLORA`	No	Lista especies, con filtro opcional por reino y el número de animales de cada una
POST · PUT · DELETE	`/api/especies[/:id]`	Sí	Crear, editar y eliminar una especie
GET	`/api/hatos`	No	Lista los hatos con su área
GET	`/api/reservas`	No	Lista las áreas protegidas
POST · PUT · DELETE	`/api/reservas[/:id]`	Sí	Crear, editar y eliminar un área
POST	`/api/reservas/:id/foto`	Sí	Subir la foto de un área
GET	`/api/actividades`	No	Lista actividades con sus horarios y cupo por horario
POST · PUT · DELETE	`/api/actividades[/:id]`	Sí	Crear (con sus 3 horarios), editar (incluido el cupo) y eliminar una actividad
POST	`/api/actividades/:id/foto`	Sí	Subir la foto de una actividad
GET	`/api/turistas`	Sí	Lista turistas con su número de reservas activas
POST · PUT · DELETE	`/api/turistas[/:id]`	Sí	Crear, editar y eliminar un turista
GET	`/api/reservas-visita/disponibilidad?fecha=AAAA-MM-DD`	No	Horarios de todas las actividades para ese día, con cupos libres y si el horario sigue vigente
GET	`/api/reservas-visita?estado=&fecha=`	Sí	Lista reservas, con filtros opcionales por estado y fecha
POST	`/api/reservas-visita`	Sí	Crear una reserva: `id_turista`, `id_horario`, `fecha_visita`, `num_personas`
PUT	`/api/reservas-visita/:id/confirmar`	Sí	Confirmar una reserva pendiente

Método	Ruta	Token	Qué hace
PUT	`/api/reservas-visita/:id/cancelar`	Sí	Cancelar una reserva, con `motivo` opcional
POST	`/api/exec/run`	Sí	Ejecutar una sentencia SQL; solo funciona con `ALLOW_EXECUTE=true`

Las fotos se sirven desde `/uploads/...`. Cuando una regla de negocio o una restricción de Oracle rechaza una operación, la API responde con código 400 y un mensaje en español; por ejemplo, ORA-00001 se traduce en "El turista ya tiene una reserva activa en ese horario y fecha".

Instalación, configuración y seguridad

Para poner el sistema en marcha se crea la base con un script, se configura `backend/.env` y se levantan dos servidores.

1. **Base de datos.** Detener el backend y cerrar las conexiones de los usuarios del proyecto. En SQL Developer, conectado como SYS con rol SYSDBA, ejecutar `proyecto_completo.sql` con F5. El script borra y recrea todo, así que se pierden los datos cargados desde la aplicación. Los `DROP` iniciales pueden dar "no existe" la primera vez.
2. **Backend.** Copiar `backend/.env.example` a `backend/.env`, completarlo, ejecutar `npm install` y luego `npm run dev` en `backend/`. Debe mostrar `Server listening on port 4000`.
3. **Frontend.** Ejecutar `npm install` y `npm run dev` en `frontend/`, y abrir `http://localhost:5173`. Vite reenvía `/api` y `/uploads` al puerto 4000.

Variables de `backend/.env`:

Variable	Obligatoria	Uso
`ORACLE_USER`, `ORACLE_PASSWORD`	Sí	Usuario de Oracle del backend: `admin_rnpg`
`ORACLE_CONNECT_STRING`	Sí	Por ejemplo `localhost:1521/XEPDB1`
`ADMIN_USER`, `ADMIN_PASS`	Sí	Usuario y clave para iniciar sesión en la aplicación web; no tienen relación con Oracle

Variable	Obligatoria	Uso
`JWT_SECRET`	Sí, mínimo 16 caracteres	Clave con la que se firman los tokens de sesión
`PORT`	No (4000)	Puerto de la API
`ALLOW_EXECUTE`	No (false)	Activa la consola SQL del panel Admin

Medidas de seguridad:

- El backend no arranca si falta alguna variable obligatoria, así que no existen claves por defecto conocidas.
- Toda ruta que modifica datos exige un token JWT válido; el token vence a las 2 horas.
- `backend/.env` está en `.gitignore` y no se sube al repositorio.
- Se eliminaron las rutas que exponían información sensible: el script SQL completo, con contraseñas, y la configuración de autenticación.
- La consola SQL está desactivada por defecto.
- Las consultas usan variables de enlace (`:nombre`), lo que evita la inyección SQL en las rutas normales.

Limitaciones conocidas y posibles mejoras

El sistema cubre el flujo completo de reservas, pero quedan estos puntos abiertos:

Punto	Situación actual	Posible mejora
Personal de la reserva	PERSONAL y ACTIVIDAD_PERSONAL tienen datos, pero no pantalla	Pantalla para registrar personal y asignarlo a actividades
Auditoría de animales	AUDITORIA_ANIMAL existe, pero nadie escribe en ella	Insertar el registro desde el backend en la misma transacción que crea, edita o borra el animal
Visitas pasadas	Una reserva cuya visita ya ocurrió se queda en PENDIENTE o CONFIRMADA	Un estado REALIZADA o NO ASISTIÓ que se marque después de la visita
Pagos	Se calcula `monto_total`, pero no se registra ningún pago	Tabla de pagos ligada a RESERVA_VISITA

Punto	Situación actual	Posible mejora
Duración frente a horarios	Hay 3 horas entre turnos y el Chaccu Tradicional dura 240 minutos	Validar que la duración no supere 180 minutos
Cuentas de acceso	Hay un solo usuario administrador, definido en `.env`	Tabla de usuarios con roles dentro de la aplicación
Tamaño de fotos	El límite de 5 MB solo se revisa en la pantalla	Limitar el tamaño también en el backend
Privilegios `ANY TABLE`	rol_aplicacion y rol_consulta pueden leer tablas de cualquier esquema	Dejar solo los permisos por objeto
Dependencias	`npm install` reporta 11 vulnerabilidades en librerías existentes	Revisar con `npm audit` y actualizar con pruebas
Nombres comunes	Los de la fauna son referenciales y la flora no tiene	Revisarlos con una fuente oficial, como SERNANP
Pruebas	No hay pruebas automáticas	Pruebas de la API para las reglas de cupo y de estado